Travaux pratiques
Chatbot basé sur un RAG

Retrieval-Augmented Generation sur un corpus Wikipedia

Durée indicative	4 heures (+ bonus)
Langage	Python 3.10 ou plus
Bibliothèques	LangChain, Chroma, OpenAI ou Mistral AI, Streamlit
Prérequis	Python, notions d'API REST, ligne de commande, Git (bases)

Contexte : qu'est-ce qu'un RAG ?

Un LLM ne connaît que ce qu'il a vu pendant son entraînement : il ignore les documents privés, les informations récentes, et peut « halluciner » des réponses plausibles mais fausses. Le principe du **RAG** (*Retrieval-Augmented Generation*) est de **rechercher** d'abord les passages pertinents dans une base documentaire, puis de les **fournir au LLM** dans le prompt pour qu'il formule une réponse fondée sur ces sources.

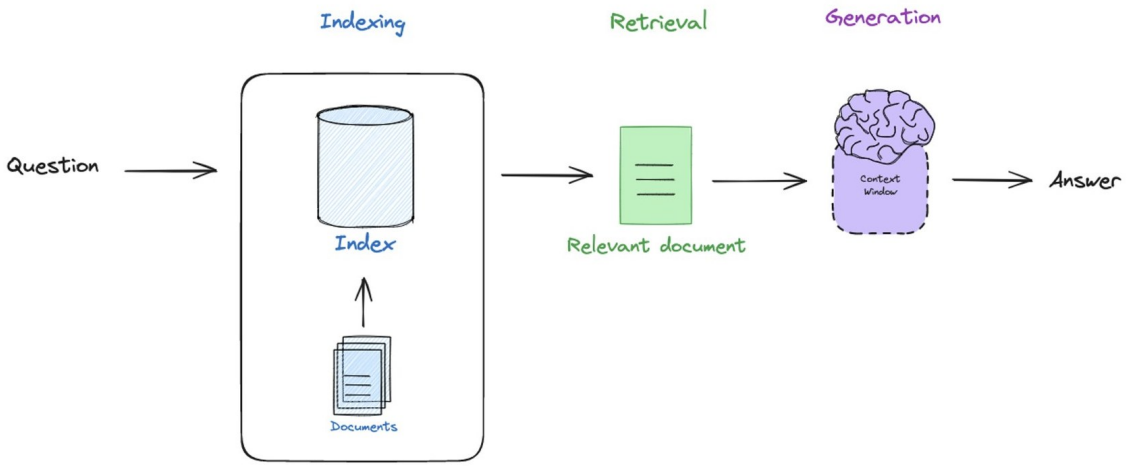


Figure 1 – Les trois étapes d'un RAG : *Indexing* (indexation des documents), *Retrieval* (recherche des documents pertinents pour la question) et *Generation* (le LLM répond à partir de ces documents placés dans sa fenêtre de contexte).
Source : support de cours « LLM – RAG » (schémas d'après LangChain, *RAG From Scratch*).

Concrètement, un RAG se déroule en trois temps :

1. **Indexation** (une seule fois) : les documents sont chargés, découpés en passages (*chunks*), transformés en vecteurs (*embeddings*) et rangés dans une base vectorielle ;
2. **Recherche** (à chaque question) : la question est elle aussi transformée en vecteur, et la base renvoie les k passages dont les vecteurs sont les plus proches ;
3. **Génération** : ces passages sont insérés dans le prompt, avec la question, et le LLM rédige la réponse.

Chaque partie du TP correspond à l'une de ces briques : chargement (partie 1), découpage (partie 2), embeddings et base vectorielle (partie 3), génération (partie 4).

Dans ce TP, la base documentaire est constituée de **12 pages Wikipedia en français** portant sur l'IA générative (ChatGPT, LLaMA, Stable Diffusion, GAN, etc.).

Question 1. [**CR**] En quelques phrases, expliquez pourquoi on ne peut pas simplement donner les 12 pages entières au LLM à chaque question. Citez au moins deux raisons.

Mise en place de l'environnement

Le code de départ contient :

Fichier	Rôle
`main.py`	code du RAG, fonctions à compléter
`requirements.txt`	dépendances Python
`.env.template`	modèle du fichier de configuration (clé API)
`README.md`	instructions d'installation

Question 2. Créez un environnement virtuel et installez les dépendances :

```
python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate
pip install -r requirements.txt
```

Le TP fonctionne avec deux fournisseurs de modèles, au choix :

Fournisseur	Clé API	Modèle de chat	Embeddings
OpenAI	fournie par l'enseignant	`gpt-6-luna`	`text-embedding-3-small`
Mistral AI	gratuite, à créer vous-même	`mistral-small-latest`	`mistral-embed`

Le code de départ choisit automatiquement le fournisseur selon la clé présente dans `.env` (OpenAI en priorité). Les fonctions `get_embeddings()` et `get_llm()` vous sont **fournies** : elles renvoient le modèle d'embeddings et le modèle de chat du fournisseur choisi. Utilisez-les dans tout le TP plutôt que d'instancier directement une classe OpenAI ou Mistral.

Question 3. Créez votre fichier `.env` :

```
cp .env.template .env
```

puis renseignez **une** des deux clés :

— `OPENAI_API_KEY` : la clé communiquée par l'enseignant ;

— **ou** `MISTRAL_API_KEY` : une clé gratuite que vous créez vous-même (voir l'encadré ci-dessous).

Créer une clé Mistral gratuite

1. Créez un compte sur `https://console.mistral.ai`.
2. Dans le menu *La Plateforme* > *Subscription*, cliquez sur *Compare plans* puis sur **Experiment for free**. Acceptez les conditions et cliquez sur *Subscribe*. Aucune carte bancaire n'est demandée, mais un numéro de téléphone vérifié est nécessaire (un seul compte par numéro).
3. Dans la section *API Keys*, créez une nouvelle clé et copiez-la immédiatement : elle n'est affichée qu'une seule fois.

Avec l'offre gratuite, le débit est limité : la première indexation peut prendre quelques minutes (le code réessaie automatiquement en cas de refus temporaire). Les requêtes peuvent aussi être utilisées par Mistral pour entraîner ses modèles : n'envoyez pas de données personnelles ou confidentielles.

Question 4. Lancez `python main.py`. Quel fournisseur et quels modèles sont affichés au démarrage ? (Le programme s'arrête ensuite avec une erreur `NotImplementedError` : c'est normal, les fonctions restent à écrire.)

Attention

Le fichier `.env` contient un **secret**. Il ne doit jamais être partagé, ni envoyé sur GitHub (voir la partie 7). La clé OpenAI de l'enseignant est commune à toute la promotion : ne la diffusez pas et ne l'utilisez que pour ce TP.

Partie 1 — Collecte des documents

La fonction `get_wikipedia_page(title)` vous est fournie : elle interroge l'API de Wikipedia et renvoie le texte brut d'une page.

Question 5. Dans un interpréteur Python, testez-la sur la page `"ChatGPT"`. Quelle est la longueur du texte obtenu ? Comment les titres de sections apparaissent-ils dans le texte ?

Question 6. Complétez la fonction `load_documents()`. Elle doit renvoyer une liste de `Document`, un par page, avec :

- `page_content` : le texte de la page ;
- `metadata` : un dictionnaire contenant le titre lisible (`"title"`, sans les `_`) et l'URL de la page (`"source"`).

Votre fonction doit ignorer (en affichant un message) les pages vides ou dont le téléchargement échoue, plutôt que de planter.

Indice

```
from langchain_core.documents import Document

doc = Document(page_content="...", metadata={"title": "...", "source": "https
    ://..."})
```

L'URL d'une page est de la forme `https://fr.wikipedia.org/wiki/<titre>`. Pour gérer les erreurs réseau, utilisez `try / except requests.RequestException`.

Partie 2 — Découpage en chunks

Une page Wikipedia peut faire plusieurs dizaines de milliers de caractères. On la découpe en morceaux d'environ 1 000 caractères, qui se chevauchent légèrement. Plusieurs stratégies de découpage existent (figure 2) : par nombre de caractères, par sections, par délimiteurs (paragraphes, phrases) ou selon le sens. `RecursiveCharacterTextSplitter` combine les deux premières : il coupe en priorité sur les délimiteurs naturels (paragraphes, puis lignes, puis phrases) tout en respectant une taille maximale.

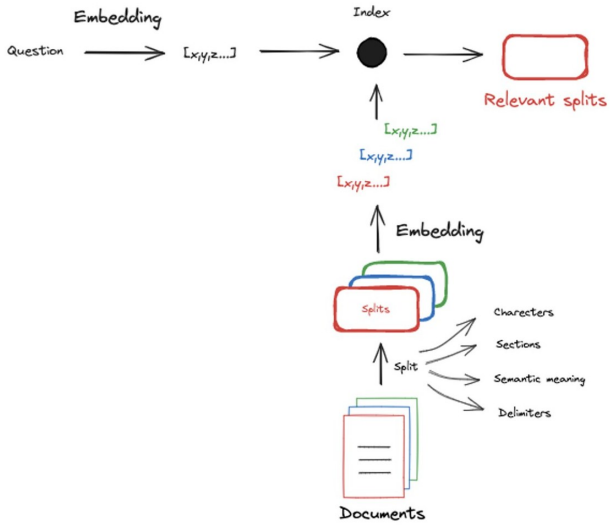


Figure 2 – Indexation : les documents sont découpés (*split*) selon différentes stratégies, chaque morceau est transformé en vecteur (*embedding*) puis stocké dans l'index. La question est vectorisée de la même façon pour retrouver les morceaux pertinents (*relevant splits*).
Source : support de cours « LLM – RAG » (schémas d'après LangChain, *RAG From Scratch*).

Question 7. Utilisez `RecursiveCharacterTextSplitter` (module `langchain_text_splitters`) avec `chunk_size=1000` et `chunk_overlap=200` pour découper la liste de documents. Combien de chunks obtenez-vous au total ?

Indice

La méthode `split_documents(documents)` renvoie une nouvelle liste de `Document`. Vérifiez que les métadonnées (titre, source) sont bien conservées dans chaque chunk.

Question 8. **[CR]** À quoi sert le chevauchement (`chunk_overlap`) entre deux chunks consécutifs ? Que risque-t-on avec des chunks trop petits ? Trop grands ?

Partie 3 — Embeddings et base vectorielle

Pour rechercher des passages, il faut une représentation numérique du texte. Deux familles existent (figure 3) :

- les représentations **statistiques** (sac de mots) : un vecteur *creux* (*sparse*) qui compte les mots, exploité par des algorithmes comme BM25 ;
- les représentations **apprises** (*embeddings*) : un vecteur *dense* de quelques centaines à quelques milliers de nombres, produit par un réseau de neurones, qui capture le *sens* du texte.

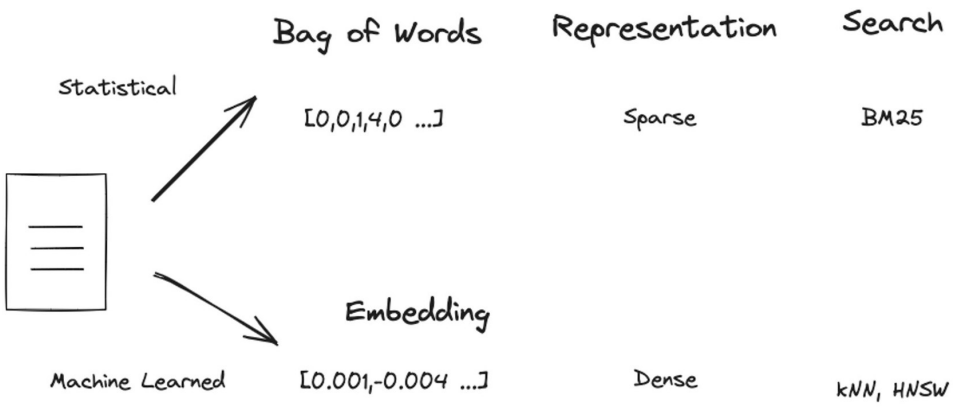


Figure 3 – Représentations statistiques (vecteur creux, recherche BM25) et apprises (vecteur dense, recherche par plus proches voisins : kNN, HNSW).
Source : support de cours « LLM – RAG » (schémas d'après LangChain, *RAG From Scratch*).

Deux textes de sens proche ont des embeddings proches. La recherche consiste donc à trouver les vecteurs de passages les plus proches du vecteur de la question, par exemple au sens de la similarité cosinus (figure 4). Une **base vectorielle** comme **Chroma** stocke ces vecteurs et réalise cette recherche efficacement grâce à un index approximatif (HNSW).

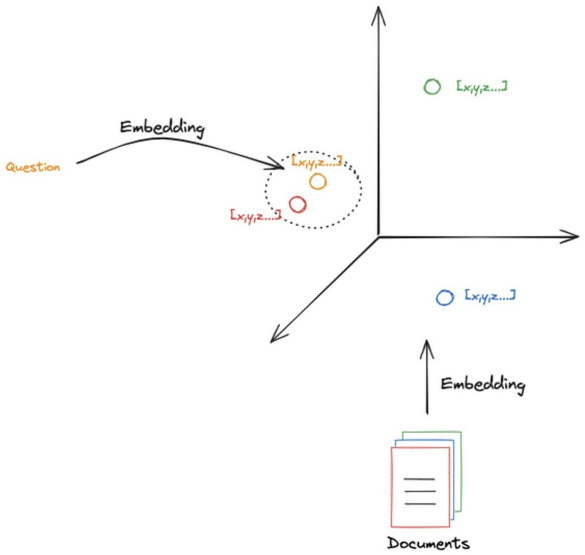


Figure 4 – Recherche par similarité : la question est projetée dans le même espace que les documents ; on retient les documents dont les vecteurs sont les plus proches.
Source : support de cours « LLM – RAG » (schémas d'après LangChain, *RAG From Scratch*).

Le modèle d'embeddings est renvoyé par la fonction fournie `get_embeddings()`.

Question 9. Dans un interpréteur, calculez l'embedding de la phrase « Qu'est-ce qu'un GAN ? » avec `get_embeddings().embed_query(...)`. Quelle est la dimension du vecteur obtenu ? **[CR]** Si des camarades utilisent l'autre fournisseur, comparez : pourquoi ne peut-on pas mélanger dans une même base des vecteurs issus de deux modèles différents ?

Question 10. **[CR]** Donnez un exemple de question pour laquelle une recherche par mots-clés (BM25) risque d'échouer alors qu'une recherche par embeddings réussira, et inversement.

Question 11. Complétez la fonction `get_retriever()` :

a) charger les documents (partie 1) puis les découper (partie 2) ;

b) créer une base `Chroma` avec `get_embeddings()` et le dossier de persistance `PERSIST_DIR` (déjà défini : `chroma_db_openai` ou `chroma_db_mistral`), puis y ajouter les chunks ;

c) renvoyer un retriever qui renvoie les $k = 4$ chunks les plus proches.

Indice

```python
from langchain_chroma import Chroma

vectorstore = Chroma(collection_name="...", embedding_function=get_embeddings(),
                     persist_directory=PERSIST_DIR)
vectorstore.add_documents(chunks)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
```

Testez votre retriever avec `retriever.invoke("Qui a créé LLaMA ?")` : les chunks renvoyés sont-ils pertinents ?

Question 12. Le calcul des embeddings prend du temps et coûte de l'argent. Modifiez `get_retriever()` pour qu'au lancement suivant, si la base existe déjà sur le disque, elle soit **rechargée** au lieu d'être recalculée.

Indice

Après avoir créé l'objet `Chroma` avec le même `persist_directory`, le nombre d'éléments déjà stockés s'obtient avec `vectorstore._collection.count()`.

Partie 4 — Génération de la réponse

Il reste à relier la recherche au LLM (figure 5). Les passages récupérés et la question remplissent les variables `{context}` et `{question}` d'un *prompt template* ; le prompt obtenu est envoyé au modèle de chat (*ChatModel*), dont la réponse est convertie en texte par un *parser*. Avec LangChain, cette chaîne s'écrit `prompt | llm | StrOutputParser()`.

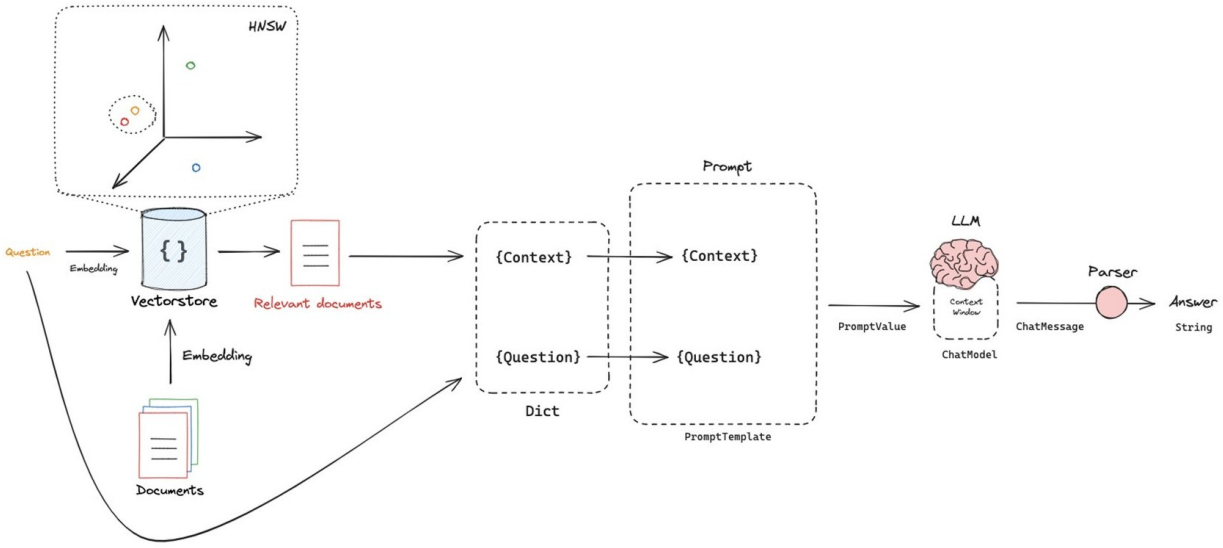


Figure 5 – Connexion de la recherche et du LLM par le prompt : {context} et {question} → *PromptTemplate* → *ChatModel* → *Parser* → réponse.
Source : support de cours « LLM – RAG » (schémas d'après LangChain, *RAG From Scratch*).

Question 13. Complétez `format_docs(docs)` : elle doit concaténer le contenu des chunks en une seule chaîne, en indiquant pour chacun le titre de la page d'origine et en les séparant clairement.

Question 14. Écrivez un `ChatPromptTemplate` composé :

— d'un message *system* qui demande au modèle de répondre en français, **uniquement** à partir du contexte fourni, et de dire qu'il ne sait pas si la réponse n'y figure pas ;

— d'un message *human* contenant la question.

Le prompt aura deux variables : `{context}` et `{question}`.

Question 15. Complétez `rag_answer(query)` :

a) récupérer les chunks pertinents avec le retriever ;

b) construire une chaîne LangChain `prompt | llm | StrOutputParser()`, où `llm = get_llm()` ;

c) afficher la réponse **au fil de l'eau** (streaming) dans le terminal ;

d) afficher ensuite la liste des sources (URL), sans doublons ;

e) renvoyer la réponse et la liste des documents utilisés (ce sera utile en partie 6).

Indice

```
for token in chain.stream({"context": ..., "question": ...}):
    print(token, end="", flush=True)
```

Pour dédoublonner une liste en conservant l'ordre : `list(dict.fromkeys(ma_liste))`.

Partie 5 — Tests du chatbot

Question 16. Lancez `python main.py` et posez les questions suivantes. Notez les réponses et les sources affichées.

(i) Qui a développé Stable Diffusion ?

(ii) Quelle est la différence entre un GAN et un modèle de diffusion ?

(iii) En quelle année ChatGPT a-t-il été lancé ?

(iv) Quelle est la capitale de l'Australie ? *(question hors corpus)*

Question 17. [**CR**] Comment le chatbot se comporte-t-il sur la question (iv) ? Retirez temporairement de votre prompt la consigne « réponds uniquement à partir du contexte » : que se passe-t-il ? Qu'en concluez-vous ?

Question 18. [**CR**] Trouvez une question portant sur le corpus à laquelle le chatbot répond mal. Affichez les chunks récupérés : le problème vient-il de la *recherche* ou de la *génération* ?

Partie 6 — Interface web avec Streamlit

Question 19. Créez un fichier `app.py` qui :

— importe `get_retriever` et `rag_answer` depuis `main.py` ;

— construit le retriever **une seule fois** (et non à chaque interaction) ;

— affiche l'historique de la conversation ;

— propose une zone de saisie de la question ;

— affiche la réponse du modèle suivie de liens cliquables vers les sources.

Lancez-le avec `streamlit run app.py`.

Indice

Fonctions Streamlit utiles :

- `@st.cache_resource` : décorateur qui exécute une fonction une seule fois et garde son résultat en mémoire ;
- `st.session_state` : dictionnaire conservé entre deux interactions (pour l'historique) ;
- `st.chat_input(...)` et `st.chat_message("user" | "assistant")` : composants de chat ;
- `st.spinner(...)`, `st.expander(...)` : indicateur de chargement, zone repliable.

Streamlit ré-exécute tout le script à chaque interaction : c'est pourquoi le cache et `session_state` sont indispensables. Pensez à adapter `rag_answer` pour pouvoir désactiver l'affichage dans le terminal (paramètre `stream=False`).

Question 20. Ajoutez une zone repliable (`st.expander`) affichant les passages récupérés pour chaque réponse.

Partie 7 — Versionner le projet avec Git

Question 21. Créez un fichier `.gitignore` à la racine du projet. Il doit exclure au minimum : le fichier `.env`, les dossiers `chroma_db_*/`, l'environnement virtuel `.venv/` et les dossiers `__pycache__/`. Justifiez **[CR]** l'exclusion de chacun.

Question 22. Créez un dépôt **vide** sur GitHub (`https://github.com/new`), puis :

```
git init
git add .
git status            # vérifiez que .env n'apparaît PAS dans la liste
git commit -m "TP RAG : premiere version"
git branch -M main
git remote add origin https://github.com/<votre-compte>/<votre-depot>.git
git push -u origin main
```

Attention

Si une clé API a été envoyée par erreur sur GitHub, la supprimer du dépôt **ne suffit pas** : elle reste dans l'historique Git. Il faut la **révoquer** immédiatement (sur la console Mistral pour votre clé, ou en prévenant l'enseignant pour la clé OpenAI) et en générer une nouvelle.

Question 23. Mettez à jour le `README.md` pour qu'une autre personne puisse installer et lancer votre projet. Faites un nouveau commit et poussez-le.

Partie 8 — Déploiement

Streamlit Community Cloud

Question 24. Déployez votre application :

1. connectez-vous sur `https://share.streamlit.io` avec votre compte GitHub ;
2. cliquez sur **Create app**, puis **Yup, I have an app** ;
3. indiquez votre dépôt, la branche `main` et le fichier principal `app.py` ;
4. dans **Advanced settings**, choisissez Python 3.12 et renseignez vos secrets au format TOML :

```
MISTRAL_API_KEY = "votre-cle-mistral"
```

5. cliquez sur **Save** puis **Deploy**.

Attention

Une application déployée est **publique** : toute personne disposant du lien consomme les crédits de la clé. Pour le déploiement, utilisez **votre propre clé Mistral gratuite**, et non la clé OpenAI commune de la promotion.

Question 25. En ligne, il n'y a pas de fichier `.env` : la clé est accessible dans le code via `st.secrets["MISTRAL_API_KEY"]`. Modifiez `app.py` pour que, si ce secret (ou `OPENAI_API_KEY`) existe, il soit copié dans `os.environ` **avant** l'import de `main.py`. Votre application doit continuer à fonctionner en local avec le `.env`.

Indice

En local, en l'absence de fichier `.streamlit/secrets.toml`, l'accès à `st.secrets` lève une exception : entourez-le d'un `try / except`.

Question 26. Modifiez une phrase de l'interface, faites un `git push` et observez ce qui se passe sur l'application déployée.

Question 27. **[CR]** Votre application est publique. Quels risques cela pose-t-il (coûts, abus) ? Proposez au moins deux mesures pour les limiter.

Alternative : conteneur Docker

Question 28 (bonus). Écrivez un `Dockerfile` basé sur `python:3.12-slim` qui installe les dépendances, copie le code, expose le port 8501 et lance Streamlit sur `0.0.0.0`. Testez-le :

```
docker build -t tp-rag .
docker run -p 8501:8501 -e MISTRAL_API_KEY=... tp-rag
```

Pourquoi faut-il ajouter un fichier `.dockerignore` ?

Pour aller plus loin

Le RAG réalisé dans ce TP est volontairement simple. De nombreuses techniques permettent de l'améliorer à chaque étape (figure 6) : reformuler la question (*query translation*), choisir la bonne source (*routing*), interroger des bases SQL ou des graphes (*query construction*), affiner l'indexation, reclasser les résultats (*re-ranking*), ou laisser le modèle relancer une recherche (*active retrieval*).

Ces questions sont facultatives. Traitez-en au moins deux et présentez vos observations dans le compte rendu.

Question 29 (bonus). Reformulation de la question (*Multi-query / RAG-Fusion*). Demandez au LLM de générer 3 reformulations de la question de l'utilisateur, lancez une recherche pour chacune, puis fusionnez les résultats (sans doublons, ou en les reclassant par *Reciprocal Rank Fusion*). Les réponses sont-elles meilleures sur vos questions de référence ?

Question 30 (bonus). Influence des paramètres. Préparez une liste de 5 questions de référence. Comparez la qualité des réponses pour `chunk_size` $\in \{300, 1000, 2000\}$ et $k \in \{2, 4, 8\}$. Présentez vos résultats sous forme de tableau.

Question 31 (bonus). Diversité des résultats. Remplacez la recherche par similarité par une recherche MMR (*Maximal Marginal Relevance*) via `as_retriever(search_type="mmr", ...)`. Qu'est-ce que cela change ?

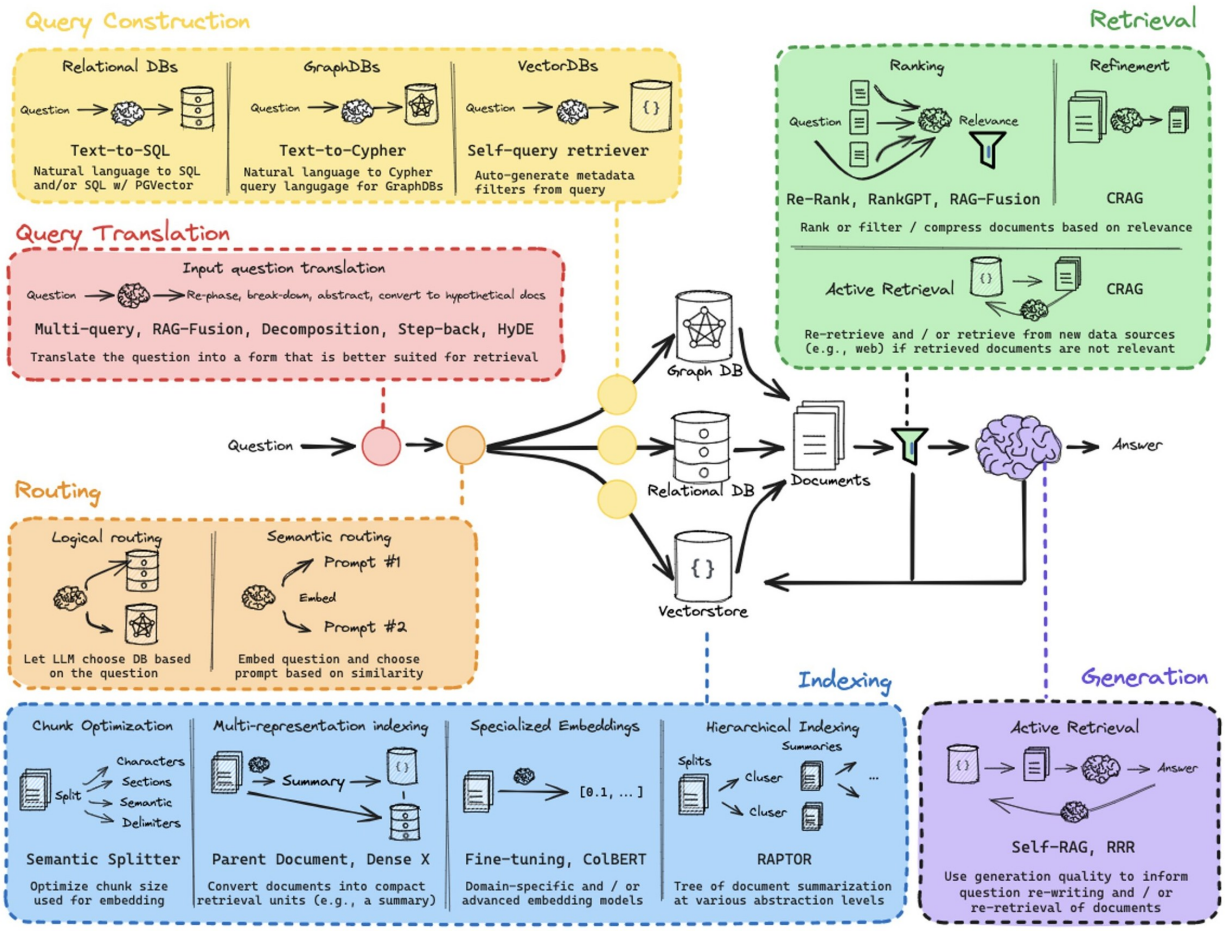


Figure 6 – Panorama des techniques de RAG avancé.
Source : support de cours « LLM – RAG » (schémas d'après LangChain, *RAG From Scratch*).

Question 32 (bonus). Mémoire conversationnelle. Actuellement, la question « Et qui l'a créé ? » posée après « Qu'est-ce que LLaMA ? » n'est pas comprise. Proposez et implémentez une solution (par exemple, reformuler la question à l'aide de l'historique avant la recherche).

Question 33 (bonus). Citations précises. Modifiez le prompt pour que le modèle indique, après chaque affirmation, le numéro du passage utilisé, par exemple `[1]`.

Question 34 (bonus). Coûts. Estimez le nombre de tokens envoyés au LLM pour une question (bibliothèque `tiktoken`). Quel est le coût approximatif de 1 000 questions ? De l'indexation ?

Question 35 (bonus). Comparaison des fournisseurs. Posez vos 5 questions de référence avec OpenAI puis avec Mistral (variable `LLM_PROVIDER` dans `.env`). Comparez la pertinence des passages récupérés, la qualité des réponses et le temps de réponse.

Question 36 (bonus). Modèles locaux. Remplacez OpenAI ou Mistral par des modèles exécutés localement (par exemple via Ollama, ou des embeddings Hugging Face). Comparez qualité et rapidité.

Question 37 (bonus). Évaluation. Proposez une méthode pour évaluer automatiquement la qualité d'un RAG (pertinence des passages récupérés, fidélité de la réponse au contexte). Vous pouvez vous inspirer de la bibliothèque RAGAS.

Annexe — Aide-mémoire

```python
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Fournies dans main.py :
embeddings = get_embeddings()    # OpenAI ou Mistral selon le .env
llm = get_llm()
```

Concept	Rappel
Chunk	Passage de texte issu du découpage d'un document.
Embedding	Vecteur représentant le sens d'un texte.
Base vectorielle	Base qui stocke des vecteurs et retrouve les plus proches d'un vecteur donné.
Retriever	Objet qui, pour une question, renvoie les k documents les plus pertinents (`retriever.invoke(question)`).
Chaîne LCEL	Composition d'étapes avec l'opérateur `|` : `prompt	llm	parser`.
Streaming	Affichage progressif de la réponse, token par token (`chain.stream(...)`).

Documentation utile

— LangChain : `https://python.langchain.com`

— Chroma : `https://docs.trychroma.com`

— Streamlit : `https://docs.streamlit.io`

— API OpenAI : `https://platform.openai.com/docs`

— API Mistral AI : `https://docs.mistral.ai`

Livrables

À rendre à la fin du TP :

1. Le lien vers votre dépôt GitHub contenant le code complet (`main.py`, `app.py`, `requirements.txt`, `.gitignore`, `README.md`).
2. L'URL de votre application déployée.
3. Un court compte rendu (1 à 2 pages) répondant aux questions de réflexion signalées par le symbole [**CR**].